

Mattia Robuschi Caprara

Basi di Dati



Prefazione

!!ATTENZIONE!!

Questo documento è un semplice OCR delle slide del prof.

Scopo (pratico) del corso

Nel corso vedremo (parzialmente in parallelo):

1. Come progettare una base di dati sulla base delle specifiche che vengono fornite.
(progettazione concettuale progettazione logica normalizzazione)
2. Come gestire e interrogare la base di dati, una volta che è stata realizzata e "popolata".
(modello relazionale, linguaggio SQL)
3. Come gestire gli accessi concorrenti alla base di dati da parte di più utenti, ma in modo sicuro (nel senso della correttezza del risultato) (transazioni)
4. Esempi di DB non relazionali (NoSQL)

Contents

1	Basi di Dati	3
2	Architettura Client-Server	3
3	DBMS	3
3.1	Affidabilità	3
3.2	Privatezza dei dati	4
3.3	Efficienza	4
3.4	Efficacia	4
3.5	Indipendenza dei dati	4
4	Modelli dei Dati	4
4.1	Il problema fondamentale	5
4.2	Modelli logici dei dati	5
4.3	Modello relazionale	6
4.4	Schema	6
5	Progettazione di una base di dati	7
6	Interfacciarsi con una base di dati	7
6.1	Linguaggi per basi di dati	7
6.1.1	SQL: un linguaggio interattivo	7
6.1.1.1	SQL embedded in Pascal code	8
6.1.1.2	SQL embedded in PHP code	8
6.2	Interazione non testuale	8
6.2.1	Vantaggi dei DBMS	8
6.2.2	Vantaggi dei DBMS	9
7	Modello relazionale	9
7.1	Tre accezioni importanti	9
7.2	Relazione - Prodotto Cartesiano	9
7.2.1	Esempio	9
7.3	Relazioni	9
7.3.1	Esempio	10
7.4	Tupla	10
7.4.1	Perché una tupla si chiama così?	10
7.4.2	Esempio	10
7.5	Cosa significa relazione	10
7.5.1	Proprietà fondamentali	10

7.5.2	Conseguenze	10
7.6	Basi di Dati e Relazioni	10
7.6.1	Esempio	11
7.6.2	Definizione relazione	11
7.6.3	Esempio di relazione con attributi	11
7.6.4	Notazione	11
7.6.5	Esempio	11
7.7	Dominio	11
7.7.1	Esempio	12
7.7.2	Vantaggi dell'approccio basato su valori	12
7.8	Informazione Sbagliata	13
7.9	Vincoli di Integrità	13
7.9.1	Vincoli di Tupla	14

1 Basi di Dati

Informazione: notizia, dato o elemento che consente di avere conoscenza più o meno esatta di fatti, situazioni, modi di essere

Dato: ciò che è immediatamente presente alla conoscenza, prima di ogni elaborazione; (in informatica) elemento di informazione costituito da simboli che devono essere elaborati (dal Vocabolario della Lingua Italiana, Istituto dell'Enciclopedia Italiana)

Base di Dati: collezione di dati, utilizzati per rappresentare le informazioni di interesse per un sistema informativo

2 Architettura Client-Server



Figure 1: Caption

Architettura client-server (applicazione lato server)
DBMS (MySQL/MariaDB), Server Web (APACHE) e interprete PHP sono integrati nel pacchetto XAMPP, scaricabile da questo link
L'applicazione risiede sulla macchina dell'utente e può essere scritta in qualsiasi linguaggio dotato di una API per connettersi, come nel caso di un browser, a un server web (protocollo HTTP) e/o a un server SQL.

3 DBMS

Un Database Management System (DBMS) è un sistema software che si interpone fra le applicazioni e la memoria di massa in cui si trovano collezioni di dati.

La finalità di un DBMS è l'estensione delle funzionalità del file system, in modo da offrire:

- Nuove modalità di accesso ai dati
- Condivisione dei dati
- Gestione più sofisticata dei file

Le basi di dati gestite dai DBMS sono collezioni di dati:

- **Grandi** possono avere notevoli dimensioni (fino a centinaia di Terabyte) e devono quindi necessariamente risiedere nella memoria secondaria
- **Condivise** applicazioni ed utenti diversi devono potere accedere ai dati
- **Persistenti** Il tempo di vita dei dati va oltre la durata dell'esecuzione delle singole applicazioni

Un DBMS deve garantire:

- Affidabilità
- Privacy dei dati
- Efficienza
- Efficacia

3.1 Affidabilità

Un DBMS deve garantire di poter mantenere intatto il suo contenuto, anche in caso di malfunzionamento. L'integrità dei dati è affidata a procedure di backup (salvataggio) e recovery (recupero) dei dati, o alla loro duplicazione nei casi più critici.

3.2 Privatezza dei dati

Ogni utente, abilitato a utilizzare la base di dati attraverso una procedura di riconoscimento, può accedere ad insiemi limitati di dati e compiere solo certe operazioni su di essi.

3.3 Efficienza

Un DBMS deve operare e fornire risposte agli utenti in tempi accettabili, utilizzando una quantità il più possibile limitata di risorse.

L'efficienza dipende essenzialmente dalle tecniche utilizzate per l'implementazione del DBMS e dalla buona progettazione della base di dati.

Si misura (come in tutti i sistemi informatici) in termini di tempo di esecuzione (tempo di risposta) e spazio di memoria (principale e secondaria) occupato.

3.4 Efficacia

Capacità di un DBMS di rendere produttive le attività degli utenti, cioè di consentire la realizzazione di basi di dati che risolvano in modo efficace i problemi degli utenti.

Concetto generico, qualitativo e non legato a specifiche funzionalità del DBMS.

Non esistono criteri oggettivi per valutarla.

3.5 Indipendenza dei dati

Il nuovo strato che il DBMS viene a creare fra memoria di massa e applicazioni consente di conservare e gestire i dati in modo indipendente dalle applicazioni stesse.

Normalmente le applicazioni accedono a dati locali gestendoli attraverso file che appartengono alle applicazioni stesse.

In presenza di un DBMS, i dati non appartengono ad una specifica applicazione, ma le diverse applicazioni vi accedono attraverso di esso.

4 Modelli dei Dati

La progettazione di una base di dati si svolge attraverso una serie di fasi successive, identificabili come:

- Raccolta delle specifiche (identificazione dei concetti)
- Progettazione concettuale
- Progettazione logica
- Progettazione fisica (non compresa nel corso)

Ogni fase fa riferimento ad un modello dei dati che corrisponde ad un meccanismo di strutturazione dei dati attuato a soddisfare le finalità della corrispondente fase della progettazione.

Un modello di dati è costituito dai concetti sulla base dei quali i dati sono strutturati e codificati.

Ogni modello di dati fornisce meccanismi di strutturazione dati, analoghi ai costruttori di tipo dei linguaggi di programmazione.

I modelli concettuali descrivono la realtà mediante concetti astratti, ma soggetti a precise regole.

Non sono finalizzati alla rappresentazione dei dati nel calcolatore, ma a descrivere (e caratterizzare per ruolo) i concetti del mondo reale di cui i dati sono istanze.

Si usano nella fase di progettazione concettuale.

Nel corso useremo il modello Entità/Relazione.



Figure 2: Modello (concettuale) entità-relazione

Si esprimono i concetti che devono essere rappresentati dai dati come concetti indipendenti (entità) e relazioni

logiche che li collegano (relazioni) e che esistono solo in presenza delle entità.

I DBMS si differenziano in base al modello logico che utilizzano.

Quindi i modelli logici, seppure astratti come i modelli concettuali, non hanno solo finalità descrittive e di documentazione, ma riflettono la struttura con cui i dati sono organizzati all'interno del calcolatore.

4.1 Il problema fondamentale

Dato un frammento della realtà che può essere identificato e isolato rispetto al "resto del mondo":

- Descriverlo nel modo più completo possibile, identificando tutti gli elementi (concetti) che lo compongono (e che permettono di distinguerlo da tutto il resto)
- Descrivere, a sua volta, ogni elemento nel modo più corretto possibile, identificandone tutte le caratteristiche (attributi) che permettono di caratterizzarlo nel contesto considerato e ignorando quelle superflue rispetto a tale contesto

Se abbiamo identificato più concetti, questi dovranno risultare logicamente collegati fra loro: ogni concetto deve essere collegato ad almeno un altro concetto, in modo che non vi sia alcun concetto (o insieme di concetti) "isolato" dagli altri.

Se così fosse, infatti, questo costituirebbe un secondo frammento della realtà indipendente da quello considerato e non avrebbe senso descriverli insieme.

Fra i concetti, alcuni (entità) potranno essere descritti in modo indipendente dagli altri attraverso un insieme di attributi che gli sono propri e tipici, senza bisogno di fare riferimento ad altri concetti.

Ad esempio, se il contesto è la gestione di un negozio, posso descrivere un prodotto attraverso il nome, il peso, il prezzo, ecc... e posso descrivere un cliente attraverso i suoi dati anagrafici.

Però non basta, due concetti che posso descrivere in modo indipendente l'uno dall'altro come cliente e prodotto non possono costituire la descrizione completa di un contesto comune.

Per descrivere il contesto del negozio, quindi, cliente e prodotto non possono fare parte della stessa descrizione (schema) senza che esista (almeno) un altro concetto che li colleghi.

D'altra parte, ovviamente, un tale concetto non può, in alcun caso, essere indipendente e non può essere descritto senza fare riferimento ai concetti che collega.

Un concetto che collega altri concetti, e che quindi non può esistere se non esistono i concetti che collega, si chiama relazione o associazione.

Nel caso del negozio, dato il concetto di cliente e di prodotto, una possibile associazione sufficiente a completare lo schema è VENDITA

Ancora non basta, una relazione è un concetto che non è descrivibile soltanto attraverso i riferimenti agli altri concetti (anche più di due) che collega, ma è anch'essa caratterizzata da specifici attributi.

Ad esempio, la descrizione di una vendita dovrà necessariamente contenere l'informazione relativa al cliente cui viene venduto un prodotto e al prodotto che ne è oggetto, ma potrà (e dovrà) anche specificare:

- Data della vendita
- Quantità del prodotto venduto
- Prezzo unitario del prodotto o prezzo totale
- Numero della fattura in cui viene riportata (in questo caso sarà necessario definire un'entità Fattura a cui fare riferimento, quindi estendere lo schema)

4.2 Modelli logici dei dati

- **Relazionale:** Il più diffuso, basato su un modello tabellare dei dati
- **Gerarchico:** Utilizzato nei primi DBMS (anni 60), tuttora utilizzato, basato su strutture ad albero
- **Reticolare:** Estensione del modello gerarchico, basato su grafi
- **A oggetti:** Estensione del modello relazionale basato sui paradigmi OOP.
- **XML (semistrutturato):** Deriva dal modello gerarchico, ma è più flessibile

4.3 Modello relazionale

E' un modello logico: definisce tipi attraverso il costruttore relazione, che organizza i dati secondo record a struttura fissa, rappresentabili attraverso tabelle.

Es. (relazioni INSEGNAMENTO e MANIFESTO)

Corso	Titolare
Basi di Dati e Web	Cagnoni
Fondamenti di Informatica	Tomaiuolo
Ingegneria Del Software	Poggi

CdL	Materia	Anno
IET-I	Basi di Dati e Web	3
IET	Fondamenti di Informatica	1
IET-I	Ingegneria del Software	3

4.4 Schema

In ogni base di dati si possono distinguere:

- **Lo schema**, sostanzialmente invariante nel tempo. Ne descrive la struttura (si parla di aspetto intensionale).
Nell'esempio, le tabelle e le "intestazioni" delle tabelle
Analogo della definizione di una classe in un linguaggio a oggetti
- **Le istanze**, cioè i valori attuali, che possono cambiare anche molto rapidamente (si parla di aspetto estensionale).
Nell'esempio, i singoli dati contenuti in ciascuna tabella
A livello concettuale, analogo di un oggetto, come istanza di una classe, in un linguaggio a oggetti

Lo schema di una base di dati è la parte dichiarativa ed invariante della base di dati e ne definisce la struttura. Nel modello relazionale lo schema di una relazione è paragonabile alla definizione del prototipo di una funzione in C:

INSEGNAMENTO (Corso, Titolare)

è lo schema della relazione INSEGNAMENTO definita su due attributi (campi del record identificati da una etichetta che ne descrive il significato).

Le n-ple di attributi appartenenti alla relazione, a ciascuno dei quali deve essere assegnato un valore, sono dette istanze della relazione.

(Basi di dati, Cagnoni) è una istanza di INSEGNAMENTO

Gli schemi possono operare a diversi livelli di astrazione.

Dalle specifiche è possibile derivare lo **Schema concettuale** che descrive i concetti che devono essere rappresentati, i dettagli che servono per descriverli e le relazioni logiche che li collegano.

Dallo schema concettuale si deriva lo **Schema logico** che descrive la struttura dell'intera base di dati mediante il modello logico adottato dal DBMS (reticolare, gerarchico, relazionale).

Che viene poi realizzato attraverso lo **Schema interno (o schema fisico)** che implementa lo schema logico mediante strutture fisiche di memorizzazione

Nei confronti dell'utente è possibile definire anche uno **Schema esterno** che descrive la struttura di una porzione della base di dati attraverso il modello logico, dal punto di vista di una classe di utenti.

Generalmente realizzato mediante viste, relazioni derivate da quelle che costituiscono lo schema logico.

Es. (i soli corsi del curriculum Ingegneria Informatica):

CdL	Materia	Anno
IET-I	Basi di Dati e Web	3
IET-I	Ingegneria del Software	3



Figure 3: Architettura standard (ANSI/SPARC) a tre livelli per DBMS

5 Progettazione di una base di dati

I diversi tipi di schema riflettono le diverse fasi della progettazione: dalle specifiche del problema da risolvere

- **Progettazione concettuale** (dalle specifiche allo schema concettuale)
Definisce quali concetti rappresentare, per quanto riguarda sia la loro descrizione che le relazioni logiche che esistono fra di essi.
- **Progettazione logica** (dallo schema concettuale allo schema logico)
Definisce come rappresentare i concetti introdotti a livello concettuale in forma di strutture dati.
- **Progettazione fisica** (dallo schema logico allo schema fisico)
Definisce come allocare e gestire fisicamente i dati all'interno del calcolatore.

6 Interfacciarsi con una base di dati

Si può eseguire l'accesso ad una base di dati nei seguenti modi:

- Linguaggi testuali interattivi
- Comandi inclusi in estensioni di linguaggi tradizionali
- Comandi inclusi in linguaggi di sviluppo ad hoc
- Interfacce grafiche amichevoli

6.1 Linguaggi per basi di dati

- Linguaggi di definizione dei dati
Utilizzati per definire gli schemi e le autorizzazioni per l'accesso
- Linguaggi di manipolazione dei dati
Utilizzati per l'interrogazione e l'aggiornamento dei contenuti della base di dati

Alcuni linguaggi specializzati, come ad esempio SQL, presentano le caratteristiche di entrambi i tipi di linguaggio.

6.1.1 SQL: un linguaggio interattivo

Come esempio immaginiamo di interfacciarci con la seguente base di dati:

Corsi	
Corso	Aula
Sistemi	N3
Reti	N2

Aule	
Nome	Piano
N3	Terra
N2	Terra

Corso	Aula	Piano
Sistemi	N3	Terra
Reti	N2	Terra

```

1 SELECT Corso, Aula, Piano
2 FROM Aule, Corsi
3 WHERE Nome = Aula AND Piano = "Terra"
4
```


6.1.1.1 SQL embedded in Pascal code

Di seguito un esempio di SQL integrato in un linguaggio di alto livello:

```
1  write('nome della citta'??');
2  readln(citta);
3  EXEC SQL DECLARE P CURSOR FOR
4      SELECT NOME, REDDITO
5      FROM PERSONE
6      WHERE CITTA = :citta ;
7  EXEC SQL OPEN P ;
8  EXEC SQL FETCH P INTO :nome, :reddito ;
9  while SQLCODE = 0 do begin
10     write('nome della persona:', nome, 'aumento?');
11     readln(aumento);
12     EXEC SQL UPDATE PERSONE SET REDDITO = REDDITO +
13         :aumento
14         WHERE CURRENT OF P
15     EXEC SQL FETCH P INTO :nome, :reddito
16     end;
17 EXEC SQL CLOSE CURSOR P
18
```

6.1.1.2 SQP embedded in PHP code

Connessione a mySQL tramite API PHP:

```
1  <html> <body>
2  <?php
3      $user="root"; $password=""; $host="127.0.0.1"; $database="prova";
4      mysql_connect($host,$user,$password);
5      @mysql_select_db($database) or die("Unable to select database");
6      $query="SELECT * from utenti";
7      $res=mysql_query($query);
8      mysql_close();
9
10     $row = mysql_fetch_assoc($res);
11     print '<table><tr>';
12     foreach($row as $name => $value) {
13         print "<th>$name</th>";
14     }
15     print '</tr>';
16
17     while($row) {
18         print '<tr>';
19         foreach($row as $value) {
20             print "<td>$value</td>";
21         }
22         print '</tr>';
23         $row = mysql_fetch_assoc($res);
24     }
25     print '</table>';
26 ?> </body> </html>
27
```

6.2 Interazione non testuale

Come interazioni non testuali si intendono generalmente i DBMS come ad esempio Microsoft Access:



Figure 4: Realizza l'approccio detto "Query by Example"

6.2.1 Vantaggi dei DBMS

- Disponibilità dei dati a tutta una comunità

- Modello unificato e preciso della realtà di interesse
- Controllo centralizzato dei dati
- Condivisione
- Indipendenza dei dati

6.2.2 Vantaggi dei DBMS

- Prodotti costosi, complessi, che richiedono investimenti in hardware, software, personale.
- Forniscono un numero elevato di servizi, in modo integrato e difficilmente scorporabile se le esigenze dell'utente sono inferiori alle caratteristiche offerte.

7 Modello relazionale

- Proposto agli inizi degli anni '70 da Codd
- Finalizzato alla realizzazione dell'indipendenza dei dati
- Unisce concetti derivati dalla teoria degli insiemi (concetto di relazione) con una rappresentazione dei dati di tipo tabellare
- Attualmente è largamente il modello più utilizzato
- Teorizzato per separare il più possibile il livello logico dal livello fisico della descrizione dei dati: la base di dati non contiene alcun riferimento all'effettiva allocazione dei dati in memoria.
- Basato su un rigoroso modello matematico, permette un elevato grado di astrazione.
- Rappresentazione tabellare: semplice ed intuitiva.
Le relazioni ed i risultati delle operazioni sulle tabelle sono facilmente rappresentabili ed interpretabili dagli utenti.

7.1 Tre accezioni importanti

1. Relazione matematica: come nella teoria degli insiemi
2. Relazione secondo il modello relazionale dei dati
3. Relazione (dall'inglese **relationship**) di tipo **logico** fra entità (concetti descrivibili in modo indipendente dagli altri) nel modello Entità-Relazione (entity-relationship); traducibile anche con **associazione** o **correlazione** (scelta preferibile per evitare ambiguità).

7.2 Relazione - Prodotto Cartesiano

Dati due insiemi D_1 e D_2 si definisce **Prodotto Cartesiano** di D_1 e D_2 ($D_1 \times D_2$) l'insieme di tutte le possibili coppie ordinate (v_1, v_2) tali che v_1 sia un elemento di D_1 e v_2 sia un elemento di D_2 .

7.2.1 Esempio

Dati gli insiemi: $A = \{cubo, cono\}$ e $B = \{rosso, verde, blu\}$

Il prodotto cartesiano tra A e B è $\{(cubo, rosso), (cono, rosso), (cubo, verde), (cono, verde), (cubo, blu), (cono, blu)\}$

7.3 Relazioni

Una relazione matematica su due insiemi D_1 e D_2 è un sottoinsieme del prodotto cartesiano $D_1 \times D_2$.

NOTA: a livello formale gli insiemi possono essere infiniti, a livello pratico non possiamo però considerare relazioni infinite.

7.3.1 Esempio

Dati gli insiemi visti, una possibile relazione è $\{(\text{cubo}, \text{rosso}), (\text{cono}, \text{rosso}), (\text{cubo}, \text{blu})\}$

o, in forma tabellare:

Cubo	Rosso
Cono	Rosso
Cubo	Blu

7.4 Tupla

Le definizioni viste per due insiemi possono essere generalizzate a n insiemi.

Ogni riga della tabella sarà allora una n -pla ordinata di elementi (detta tupla).

n è detto **grado** del prodotto cartesiano e quindi della relazione.

Il numero di elementi (istanze) della relazione è detto **cardinalità** della relazione.

Un insieme può apparire più volte in una relazione.

7.4.1 Perché una tupla si chiama così?

Perché è una n -upla (ennupla) che viene rappresentata di solito con la lettera t anziché con la lettera n

7.4.2 Esempio

La relazione `Partite_di_Calcio(Casa, Ospiti, RetiCasa, RetiOspiti)` è un sottoinsieme del prodotto cartesiano **Stringa x Stringa x Intero x Intero**.

7.5 Cosa significa relazione

Una relazione è concetto mutuato dalla definizione di relazione matematica della teoria degli insiemi, definito come sottoinsieme del prodotto cartesiano fra n insiemi.

Nel modello relazionale corrisponde ad una struttura dati tabellare.

7.5.1 Proprietà fondamentali

- Ogni tupla è internamente ordinata: l' i -esimo valore proviene dall' i -esimo dominio (struttura posizionale)
- Non esiste ordinamento intrinseco fra le tuple, per la natura insiemistica della relazione
- Non sono ammesse tuple uguali (ogni elemento di un insieme è unico)

7.5.2 Conseguenze

- Lo scambio fra righe di una tabella non modifica la relazione
- Lo scambio fra colonne di una tabella può portare alla sua inconsistenza con lo schema

7.6 Basi di Dati e Relazioni

Uno **schema di relazione** $R(X)$ è costituito da un simbolo (nome della relazione) R e da una serie di attributi $X = A_1, A_2, \dots, A_n$

Quindi una tupla t è una istanza di una relazione r (un elemento del corrispondente insieme) la cui struttura è definita dal corrispondente schema $R(X)$.

Se la relazione è rappresentata come tabella, una istanza della relazione è una riga della tabella:

01	Analisi	Rossi
----	---------	-------

Uno **schema di base di dati** è un insieme di schemi di relazione con nomi diversi

$$R = R_1(X_1), R_2(X_2), \dots, R_n(X_n)$$

Una **base di dati** definita su uno schema

$$R = R_1(X_1), R_2(X_2), \dots, R_n(X_n)$$

è un insieme di relazioni $r = r_1, r_2, \dots, r_n$ dove ogni r_i è una relazione sullo schema $R_i(X_i)$.

A livello di rappresentazione, una base di dati è un **insieme di tabelle**.

7.6.1 Esempio

Corsi(*Codice, Titolo, Docente*)

Una relazione su uno schema $R(X)$ è un insieme r di tuple definite su X : è rappresentata come una tabella:

Corsi		
Codice	Titolo	Utente
01	Analisi	Rossi
02	Chimica	Bruni
04	Chimica	Verdi

7.6.2 Definizione relazione

Una relazione è un **insieme di record omogenei**, cioè definiti sugli stessi campi.

Come ogni campo di un record è associato ad un **nome (etichetta)**, così si associa ad ogni colonna della relazione un **attributo**.

7.6.3 Esempio di relazione con attributi

Corsi			
Casa	Ospiti	RetiCasa	RetiOspiti
Parma	Inter	3	2
Palermo	Lazio	2	0
Milan	Juventus	1	1

7.6.4 Notazione

Se t è una tupla definita sullo schema $R(X)$ (insieme ordinato di domini) di una relazione e A è uno dei domini di $R(X)$.

$t[A]$ (o $t.A$) è il valore di t relativo al dominio A .

7.6.5 Esempio

[relazione $Partite(Casa, Ospiti, RetiCasa, RetiOspiti)$]

Se t è la prima tupla della relazione (vedi pagine precedenti), allora $t.Casa = \text{Parma}$



Figure 5: Livello descrittivo e Rappresentazione

7.7 Dominio

Ogni attributo ha un suo **dominio** su cui è definito.

Quindi una tupla è un insieme di valori, uno per attributo, ordinati secondo lo schema della relazione e definiti ciascuno su un proprio dominio.

Una relazione è una serie di tuple definite sullo schema della relazione (insieme ordinato dei domini dei singoli attributi).

Studenti(<i>Matricola, Cognome, Nome, DataNascita</i>)
Corsi(<i>Codice, Titolo, Docente</i>)
Esami(<i>Studente, Voto, Corso</i>)

- **Studenti** contiene dati su un insieme di studenti.
- **Corsi** contiene dati su un insieme di corsi.
- **Esami** contiene dati su un insieme di esami e fa riferimento alle altre due attraverso i numeri di matricola e il codice del corso.



Figure 6: Schema logico base di dati

Quindi Matricola e Studente, come Corso e Codice, sono definiti sullo stesso dominio e possono (in questo caso devono, per generare il riferimento) assumere gli stessi valori. Lo schema logico della base di dati è la descrizione, a livello logico, di uno schema concettuale in cui, se si usa la rappresentazione col modello Entità/Relazione:

- Studenti e Corsi sono due entità, perché sono concetti che esistono indipendentemente dagli altri e possono essere rappresentati e descritti in modo autonomo.
- Esami è una relazione, perché è un concetto che esiste solo come collegamento (correlazione, associazione) fra due concetti rappresentabili come entità.

Importante notare:

- A livello concettuale, l'unico attributo dell'entità Esami è Voto.
Gli attributi di Esami che servono come riferimenti, cioè Studente (verso la tabella Studenti) e Corso (verso la tabella Corsi), sono attributi di Esami nel modello logico solo per tradurre l'informazione che nel diagramma E/R è fornita dal legame grafico e specifica che Esami è un'associazione che collega le entità Corsi e Studenti.
- Come attributi, Studente e Corso sono definiti su un dominio semplice (cioè ogni attributo ha un solo valore) e quindi fanno riferimento a una sola istanza di Studenti o Corsi.

7.7.1 Esempio

Studenti				Esami			Corsi		
Matricola	Cognome	Nome	Data di nascita	Studente	Voto	Corso	Codice	Titolo	Docente
6554	Rossi	Mario	05/12/1978	3456	30	04	01	Analisi	Mario
8765	Neri	Paolo	03/11/1976	3456	24	01	02	Chimica	Bruni
9283	Verdi	Luisa	12/11/1979	9283	28	01	04	Chimica	Verdi
3456	Rossi	Maria	01/02/1978	6554	26	02			

Dall'esempio abbiamo visto che:



Figure 7: Esplicitazione relazioni fra le tabelle

- Il modello relazionale è basato su valori.
- I riferimenti fra dati in relazioni diverse avvengono attraverso la corrispondenza dei valori con i quali (insiemi di) attributi corrispondenti vengono istanziati nelle tuple che sono logicamente correlate.

NB Attributi corrispondenti può implicare uguaglianza del nome oppure corrispondenza (logica) fra valori ammissibili, attraverso l'imposizione di vincoli. In genere i due (insiemi di) attributi ammettono gli stessi valori oppure, per uno dei due, sono ammissibili i valori di un sottoinsieme dei valori ammissibili per l'altro.

Gli altri modelli (gerarchico, reticolare) utilizzano puntatori per realizzare la corrispondenza.

7.7.2 Vantaggi dell'approccio basato su valori

- Si inseriscono nella base di dati solo valori significativi per l'applicazione (i puntatori sono dati aggiuntivi relativi alla sola implementazione fisica).

- Il trasferimento dei dati da un ambiente ad un altro è più semplice (i puntatori hanno validità solo locale)
- La rappresentazione logica dei dati non fa riferimento a quella fisica e quindi si ottiene l'indipendenza (fisica) dei dati

7.8 Informazione Sbagliata

Le tuple che compongono la base di dati devono essere omogenee.
Quindi, in ogni tupla, ad ogni attributo deve essere associato un valore.
Non sempre questo è possibile.

Es. Persone(*Cognome, Nome, Indirizzo, Telefono*)

Potrebbe esistere una persona che non possiede telefono, o di cui non conosciamo l'indirizzo.

Cognome	Nome	Indirizzo	Telefono
Rossi	Francesco	v. Rossa 22	0521 335643
Verdi	Giuseppe	v. Verde 11	
Bruni	Bruno		336 7564213

Questa relazione non è ammissibile! In ogni tupla, ogni attributo deve essere istanziato.

Non conviene (anche se è un espediente di uso comune) usare valori del dominio normalmente non utilizzati (0, stringa nulla, "99", ...), come spesso accade nella programmazione:

- Potrebbero non esistere valori non utilizzati
- Valori non utilizzati potrebbero, a un certo punto, diventare significativi ed essere utilizzati
- In fase di utilizzo (nei programmi) sarebbe necessario ogni volta tenere conto del "significato" (non standardizzato) di questi valori

Nel modello relazionale è definito un valore convenzionale, detto valore nullo, che indica la non disponibilità dell'informazione.

Il valore nullo può rappresentare correttamente tre situazioni logicamente diverse in cui l'informazione è:

- Sconosciuta (so che il valore esiste ma non lo conosco)
- Inesistente (so che il valore non esiste)
- Indeterminata (non so se il valore esiste e, in ogni caso, non lo conosco)

7.9 Vincoli di Integrità

Non tutte le combinazioni possibili di valori dei domini su cui è definita una relazione sono accettabili.

- Alcuni attributi possono assumere valori solo in un certo intervallo
- Alcuni attributi devono essere diversi in ogni tupla della stessa relazione

Es. valori dell'attributo Matricola nella relazione
Studenti(*Matricola, Cognome, Nome, DataNascita*)

Alcuni valori possono essere incompatibili con altri all'interno della stessa relazione

Es. data la relazione
Esami(*Matricola, Voto, Lode, CodCorso*)

1. Una stessa coppia Matricola, Corso può apparire una sola volta
2. Il valore Vero per l'attributo Lode è corretto solo se Voto=30

Alcuni valori possono essere incompatibili con i valori di un'altra relazione

Es. date le relazioni:
Studenti (*Matricola, Cognome, Nome, DataNascita*)
Esami (*Studente, Voto, Lode, CodCorso*)
Corsi (*CodCorso, Titolo, Docente*)

1. Ogni valore di CodCorso in Esami, in questo caso, DEVE essere un valore esistente di CodCorso nella relazione Corsi;
2. Analogamente, ogni valore dell'attributo Studente nella tabella Esami DEVE essere un valore esistente dell'attributo Matricola nella relazione Studenti.

Sono condizioni, espresse come predicati logici, inserite nella base di dati per garantirne la consistenza.

Ogni istanza della base di dati deve soddisfare i vincoli di integrità, cioè il predicato corrispondente al vincolo deve assumere valore vero **in ogni istante**.

Una istanza che soddisfi tutti i vincoli è detta corretta (o lecita, o ammissibile)

Un vincolo è detto:

- **Intra-relazionale** se coinvolge attributi della stessa relazione

Es.

- **Vincoli di tupla** possono essere valutati su ciascuna tupla indipendentemente dalle altre
- **Vincoli di dominio** definiti su singoli valori
- **Interrelazionale** se coinvolge più relazioni

7.9.1 Vincoli di Tupla

Possono essere definiti attraverso operatori booleani

Es. Data la relazione:

Esami(*Matricola*,*Voto*,*Lode*,*CodCorso*)

```
1 (Voto >= 18) AND (Voto <= 30)
2 (NOT (lode=Vero)) OR (Voto=30)
3
```

Oppure, data la relazione:

Pagamenti(*Data*,*Importo*,*Ritenute*,*Netto*)

```
1 Netto = Importo - Ritenute
2
```

8 Continua su pacco di Slide 6